Artifical Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning**: A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Turing Test**: An operational test for intelligent behavior, proposed by Alan Turing, where a human evaluator must distinguish between a human and a machine based on their textual responses.
- **Game Tree**: A tree-like structure representing all possible sequences of moves in a game, with leaf nodes representing the outcomes.
- **Perceptron**: A fundamental unit in neural networks, consisting of weighted inputs, a summation node, and an activation function that produces an output.
- **Reinforcement Learning**: A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.
- **Matrix**: A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform**: A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c.
- **Matrix Multiplication**: An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of row i of the first matrix and column j of the second matrix. The inner dimensions must match.
- **Matrix Inverse**: For a square matrix A, its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition**: An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T.
- **Derivative**: A measure of how a function changes in response to a small change in its input. It represents the instantaneous rate of change.
- **Partial Derivative**: A derivative of a function of multiple variables with respect to one of those variables, treating the others as constants.
- **Sigmoid Function**: A mathematical function that maps any input value to an output value between 0 and 1. Often used as an activation function in neural networks.
- **Inductive Reasoning**: The process of deriving general principles from specific observations.
- **Linear Regression**: A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Model Evaluation**: The process of assessing the performance of a machine learning model.
- **Overfitting**: A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Empirical Risk Minimizer**: A machine learning approach that aims to find the model that minimizes the average loss on the training data.
- **Binary Classification**: A machine learning task where the goal is to classify data into one of two categories.
- **Decision Boundary**: A line or surface that separates different classes in a classification problem.
- **0-1 Loss**: A loss function in classification that assigns a loss of 1 if the prediction is incorrect and 0 if it is correct.

- **Perceptron Loss**: A loss function used in the perceptron algorithm, defined as max\0, -z\, where z = y(w^Tx).
- **Hinge Loss**: A loss function used in support vector machines, defined as max\0, 1 - z\, where z = y(w^Tx).
- **Logistic Loss**: A loss function used in logistic regression, defined as log(1 + e^{-z}), where z = y(w^Tx).
- **Gradient Descent**: An iterative optimization algorithm used to find the minimum of a function by following the negative gradient.
- **Stochastic Gradient Descent**: A variant of gradient descent that updates the model parameters using a randomly selected subset of the training data.
- **Logistic Regression**: A machine learning algorithm used for binary classification that models the probability of a data point belonging to a particular class using a logistic function.
- **Bag of Features**: A representation of an image as a collection of visual features (codewords) and their frequencies, used for object recognition.
- **Category Models**: Representations of different object categories in a model space, based on their extracted features.
- **Model Space**: A multi-dimensional space where each point represents a category model, allowing for object categorization based on feature vectors.
- **Edge Detection**: The process of identifying points in an image where there is a significant change in intensity.
- **Edges**: Curves in an image where brightness changes rapidly.
- **Corners/Junctions**: Points in an image where there are significant changes in intensity in multiple directions.
- **Intensity Profile**: A plot of intensity values along a horizontal scanline of an image.
- **Image Derivatives**: Approximations of the rate of change of intensity in an image, often used for edge detection.
- **Finite Difference**: A method for approximating the derivative of a function using discrete values.
- **Linear Filter**: A filter that performs a linear transformation on an image, often represented as a convolution operation.
- **Moving Average**: A smoothing filter that averages the values in a local neighborhood of pixels.
- **Image Gradient**: A vector that points in the direction of the most rapid increase in intensity, with its magnitude representing the rate of change.
- **Canny Edge Detector**: An algorithm for edge detection that uses multiple steps, including gradient calculation, non-maximum suppression, and hysteresis thresholding.
- **Vision Pyramids**: Hierarchical representations of an image at multiple resolutions, used for feature extraction at different scales.
- **Gaussian Pyramids**: A type of vision pyramid that uses Gaussian blurring and subsampling to create a hierarchy of image representations.
- **Convolution**: A mathematical operation that combines two functions to produce a third function, often used in image processing for filtering and feature extraction.
- **Harris Detector**: A corner detection algorithm that uses the second moment matrix to identify corners in an image.
- **Harris Features**: Points in an image identified as corners by the Harris detector.
- **Corner Detection**: The process of identifying points in an image where there are significant changes in intensity in multiple directions.
- **Sum of Squared Differences (SSD)**: A metric used in corner detection to measure the difference in pixel intensities between a window and its shifted version.
- **Derivative of Gaussian Filter**: A filter used for edge detection that is created by taking the derivative of a Gaussian function.
- **Anisotropic Gaussian**: A Gaussian function with different standard deviations along the x and y axes.
- **Isotropic Gaussian**: A Gaussian function with the same standard deviation in all directions.
- **Convolutional Neural Network (CNN)**: A type of artificial neural network designed to process data with a grid-like topology, such as images. CNNs use convolutional layers to extract features from the input data, followed by pooling layers to reduce the dimensionality of the data.

- **ImageNet**: A large dataset of images used for training and evaluating image recognition models. It contains millions of images across thousands of categories.
- **AlexNet**: One of the first deep convolutional neural networks to achieve high accuracy in image recognition competitions, notably the ImageNet challenge. It was a significant milestone in the development of deep learning.
- **Dense Neural Network**: A type of artificial neural network where each neuron in one layer is connected to every neuron in the next layer. This creates a fully connected network.
- **ConvNet**: Short for Convolutional Neural Network.
- **Artificial Neuron**: A computational model inspired by biological neurons. It takes weighted inputs, sums them, and applies an activation function to produce an output.
- **Convolution Layer**: A layer in a convolutional neural network that applies a filter (kernel) to the input data to extract features. The filter slides across the input, performing element-wise multiplication and summation at each position.
- **Pooling Layer**: A layer in a convolutional neural network that reduces the spatial dimensions of the feature maps by applying a pooling function, such as max pooling or average pooling.
- **Max Pooling**: A type of pooling operation where the maximum value within a defined region (e.g., 2x2) is selected as the output.
- **Average Pooling**: A type of pooling operation where the average value within a defined region (e.g., 2x2) is selected as the output.
- **Local Receptive Field**: The small region of the input data that a neuron in a convolutional layer is connected to. This leads to sparse connectivity.
- **Sparse Connectivity**: A characteristic of convolutional neural networks where each neuron is connected to only a few neurons in the previous layer, unlike fully connected networks.
- **Parameter Sharing**: A key characteristic of convolutional layers where the same set of weights (filter) is applied across different spatial locations in the input.
- **ReLU**: Short for Rectified Linear Unit, a non-linear activation function defined as $f(x) = \max(0, x)$.
- **Back-propagation**: An algorithm used to train artificial neural networks by calculating the gradient of the loss function with respect to the network's weights and updating the weights accordingly.
- **Fully Connected Layer (FC)**: A layer in a neural network where each neuron is connected to every neuron in the previous layer.
- **MNIST**: A large database of handwritten digits used for training and testing in machine learning, particularly image recognition.
- **CIFAR-10**: A large dataset of 32x32 color images, commonly used for training and evaluating image classification models.
- **LeNet-5**: A convolutional neural network architecture specifically designed for the MNIST dataset.
- **Training Data**: The subset of a dataset used to train a machine learning model.
- **Held Out Data**: A subset of the training data used for hyperparameter tuning and model selection.
- **Test Data**: A subset of the dataset used to evaluate the performance of a trained machine learning model on unseen data.
- **Generalization**: The ability of a machine learning model to perform well on unseen data.
- **Underfitting**: A phenomenon where a model is too simple to capture the underlying patterns in the data, resulting in poor performance on both training and unseen data.
- **Bias**: The error introduced by approximating a real-world problem with a simplified model.
- **Variance**: The error introduced by the model's sensitivity to small fluctuations in the training data.
- **Hyperparameters**: Parameters that control the learning process of a machine learning model, but are not learned from the data itself.
- **Regularization Coefficient (λ)**: A hyperparameter that controls the strength of the regularization penalty.
- **Learning Curves**: Graphs that show the training and testing performance of a model as a function of the training set size or model complexity.
- **S-fold Cross Validation**: A model evaluation technique that divides the data into S folds, using each fold as a validation set once while training on the remaining folds.
- **Data Analysis Pipeline**: A sequence of steps involved in analyzing data, including data collection, model design, parameter inference, and model application.

- **Matrix Factorization**: A technique to decompose a large matrix into the product of two smaller matrices, revealing latent factors that capture underlying relationships in the data.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction technique that transforms data into a lower-dimensional space while maximizing variance. It finds the principal components, which are orthogonal directions of maximum variance in the data.
- **Dimensionality Reduction**: Techniques that transform data from a higher-dimensional space to a lower-dimensional space, reducing the number of attributes while preserving important information.
- **Low Rank Matrix Approximations**: Approximating a high-dimensional matrix with a lower-rank matrix, effectively reducing the dimensionality of the data while preserving essential information.
- **Eigenvectors**: Vectors that, when multiplied by a matrix, only change in scale (not direction). In PCA, they represent the principal components.
- **Eigenvalues**: The scaling factors associated with eigenvectors. In PCA, they represent the variance explained by each principal component.
- **Embeddings**: A low-dimensional representation of data points, often used to project high-dimensional data into a lower-dimensional space for visualization or analysis.
- **Collaborative Filtering**: A recommender system technique that predicts user preferences based on the ratings of similar users.
- **Recommender Systems**: Systems that predict user preferences for items, such as movies or products, based on past behavior and similar users' preferences.
- **Root Mean Square Error (RMSE)**: A metric used to evaluate the accuracy of a recommender system by calculating the square root of the average squared differences between predicted and actual ratings.
- **Bias-Variance Tradeoff**: The balance between model complexity and prediction accuracy. High complexity can lead to overfitting (high variance), while low complexity can lead to underfitting (high bias).
- **Single Nucleotide Polymorphisms (SNPs)**: Variations in a single nucleotide base (A, C, G, or T) in a DNA sequence, commonly used in genetic studies.
- **Clustering**: The process of grouping a set of objects into classes of similar objects. Objects within a cluster should be similar, and objects from different clusters should be dissimilar.
- **K-means clustering**: A commonly used clustering algorithm that partitions objects into K disjoint subsets, assuming the objects are p-dimensional vectors and using a distance measure (typically Euclidean distance) between them.
- **Euclidean distance**: The most commonly used distance measure for continuous data, defined as $d_E(x, y) = \sqrt{\sum_{i=1}^{p}(x_i - y_i)^2}$, where p is the number of attributes/dimensions.
- **Hamming distance**: A distance measure used for bit/binary vectors that counts the number of bits that differ. It can be generalized to discrete data to count mismatches.
- **Matching coefficient**: A normalized Hamming distance, obtained by dividing the Hamming distance by the number of attributes/bits.
- **Inertia**: In the context of K-means, the sum of squared distances from each point to its cluster centroid. Minimizing inertia is the goal of K-means optimization.
- **Purity**: A measure of the extent to which clusters contain objects of a particular class. Calculated as $\text{purity}(C, G) = \frac{1}{N}\sum_{i=1}^{K}\max_j N_j \in G_i$, where N is the total number of points, K is the number of clusters, and N_j is the number of examples of class j in cluster i.
- **Intelligent Agent**: A system that perceives its environment and takes actions that maximize its chances of success.
- **Logic Reasoning**: A cornerstone of Artificial Intelligence; the process of using rules of deduction to derive a conclusion from given assumptions.
- **Satisfiability Problem (SAT)**: The problem of determining whether a given propositional formula is satisfiable, i.e., whether there exists an assignment of truth values to its variables that makes the formula true.
- **3-Satisfiability (3-SAT)**: A restricted version of the satisfiability problem where each clause in the propositional formula contains exactly three literals.

- **Model Checking**: A technique used to verify the correctness of hardware and software systems by checking if a system can reach an undesirable state or if it will always reach a desired state.
- **Classical Planning**: A type of planning problem that involves finding a sequence of actions to achieve a goal, often represented using a planning graph and encoded as a CNF formula for solving with SAT solvers.
- **Combinatorial Design**: The creation of complex mathematical structures with desirable properties, such as Latin squares, quasi-groups, Ramsey numbers, and Steiner systems, often solved using SAT solvers.
- **Conjunctive Normal Form (CNF)**: A standardized way of representing logical expressions as a conjunction of clauses, where each clause is a disjunction of literals.
- **Literal**: A propositional variable or its negation.
- **Clause**: A disjunction of literals.
- **Equisatisfiable**: Two formulas are equisatisfiable if they have the same set of satisfying assignments.
- **Recursion**: A process where a function calls itself, eventually terminating in a base case.
- **Base Case**: The simplest form of a problem in a recursive approach, providing a direct solution without further recursion.
- **Recursive Case**: In a recursive approach, the same problem/method/data structure, but smaller, closer to the base case.
- **Breadth First**: A tree traversal method that visits nodes level by level.
- **Depth First**: A tree traversal method that traverses down a branch of a tree.
- **Binary Search Tree**: A tree data structure where each node has a value, its left child has a smaller value, and its right child has a larger value.
- **Problem Solving as Search**: An approach to problem-solving that involves exploring possibilities to find a solution.
- **States**: Descriptions of the configuration of an environment in a search problem.
- **Actions**: Activities that move from one state to another in a search problem.
- **Search Algorithm**: Determines which actions should be tried in which order to find a solution in a search problem.
- **Successor Function**: A function that maps a state to its possible successor states.
- **Dynamic Programming**: An optimization technique for recursive algorithms that avoids redundant calculations by storing and reusing previously computed values.
- **Graph**: A data structure consisting of nodes and edges, where edges connect pairs of nodes.
- **Rational Agent**: An agent whose actions are expected to maximize goal achievement, given available information.
- **General Automated Reasoning**: A process that uses a generic model generator to create a model from a problem instance, and then uses a general inference engine to find a solution. This approach aims to improve reasoning and modeling technology across various domains.
- **Exponential Complexity Growth**: The phenomenon where the computational complexity of a problem increases exponentially with the size of the input. This makes solving large instances of many problems computationally infeasible.
- **Bounded Model Checking (BMC)**: A formal verification technique that checks if a given model satisfies a temporal property within a bounded number of steps. It reduces the problem to a propositional satisfiability problem.
- **Partial Assignment**: An assignment of truth values to a subset of the variables in a Boolean formula.
- **Simplification of a formula**: The process of reducing a Boolean formula by removing clauses that are trivially true or literals that are trivially false, based on a partial assignment.
- **Davis-Putnam-Logemann-Loveland (DPLL) Algorithm**: A complete and backtracking-based algorithm for solving the Boolean satisfiability problem.
- **Pure Literal Rule**: A rule in SAT solving that states that if a variable appears only positively or only negatively in a formula, it can be assigned a truth value that satisfies all clauses containing it.
- **Unit Propagation (BCP)**: A technique in SAT solving that efficiently assigns truth values to variables based on unit clauses (clauses with only one literal).
- **Resolution Rule**: An inference rule in propositional logic that allows deriving a new clause from two clauses containing a literal and its negation.

- **Clause Learning**: A technique in SAT solving where clauses are learned from conflicts during the search process to avoid repeating similar mistakes.
- **1-UIP Scheme**: A method for selecting the learned clause in clause learning, focusing on the unique implication point (UIP) closest to the conflict.
- **Heavy-tailed Behavior**: The phenomenon where the runtime distribution of a SAT solver is heavily skewed, with a few instances taking significantly longer than the average.
- **Hamiltonian Path Problem**: A graph problem that asks whether there exists a path in a graph that visits each vertex exactly once.
- **Graph Coloring**: The problem of assigning colors to the vertices of a graph such that no two adjacent vertices have the same color.
- **Cook's Theorem**: A fundamental theorem in computational complexity theory that states that the Boolean satisfiability problem (SAT) is NP-complete.
- **Fairness**: The property of an algorithm or system to treat all individuals or groups equitably, without discrimination.
- **Disparate Treatment**: A type of discrimination where a decision-making process explicitly uses protected attributes (e.g., race, gender) to make decisions.
- **Disparate Impact**: A type of discrimination where a decision-making process, even without explicitly using protected attributes, disproportionately harms or benefits certain groups.
- **Privacy**: The right of individuals to control their personal information and how it is used.
- **Security**: The protection of data from unauthorized access, use, disclosure, disruption, modification, or destruction.
- **Algorithmic Opacity**: The difficulty in understanding how an algorithm arrives at its decisions.
- **Proxies for Discrimination**: Seemingly neutral variables that correlate with protected attributes and can indirectly lead to discriminatory outcomes.
- **Interpretable Results**: The ability to understand and explain the reasoning behind an algorithm's decisions.
- **Group Fairness**: A fairness criterion that requires equalizing outcomes across different groups.
- **Individual Fairness**: A fairness criterion that requires similar individuals to be treated similarly.
- **Automatic Prediction System**: A system that uses algorithms to make predictions automatically.
- **Data**: A collection of facts, figures, or information used for analysis.
- **Model**: A mathematical representation of a system or process used to make predictions or decisions.
- **Population Samples**: A subset of the population used to train a machine learning model.
- **True Population**: The entire population that a machine learning model is intended to make predictions about.

2 Problem Types

- **Understanding the Nature of Intelligence**: This problem explores the definition and characteristics of intelligence, both human and artificial. It examines various perspectives on what constitutes intelligence and how it can be measured or simulated.
- **Defining Artificial Intelligence**: This problem involves formulating a precise definition of artificial intelligence, considering different perspectives and historical contexts. The challenge lies in capturing the essence of AI while accounting for its evolving nature and diverse applications.
- **Passing the Turing Test**: This problem focuses on the implications of passing the Turing Test, a benchmark for machine intelligence. It explores whether passing the test truly indicates machine intelligence or merely the ability to mimic human behavior.
- **Overcoming the Limitations of Search**: This problem addresses the computational challenges associated with exhaustive search in complex games like chess and Go. It explores the need for alternative approaches, such as machine learning, to overcome the limitations of brute-force search.
- **Addressing the Challenges of Computer Vision**: This problem investigates the difficulties in developing computer vision systems that can reliably interpret and understand images. It highlights the gap between initial expectations and the ongoing challenges in this field.
- **Mitigating AI Safety Risks**: This problem examines the potential risks associated with AI systems,

such as adversarial attacks and biases. It explores methods for improving AI safety and ensuring responsible development and deployment.

- **Addressing Ethical Concerns in AI**: This problem focuses on the ethical implications of AI systems, particularly concerning bias and fairness. It explores the need for responsible AI development and deployment that considers societal impact and avoids perpetuating harmful biases.
- **Understanding the Differences Between Game Types**: This problem explores the differences between various game types, such as chess and soccer, in the context of AI. It examines how these differences affect the design and implementation of AI algorithms for playing these games.
- **Bridging the Gap Between Different AI Domains**: This problem investigates the similarities and differences between seemingly disparate AI domains, such as speech synthesis and game playing. It explores the potential for transferring knowledge and techniques across different AI applications.
- **Developing Robust and Generalizable Algorithms**: This problem focuses on the design of algorithms that can adapt to new situations and generalize their knowledge to unseen scenarios. It explores the challenges in creating algorithms that are not only effective in specific tasks but also robust and adaptable.
- **Understanding the AI Grand Challenges**: This problem explores the reasons behind selecting specific AI problems, such as autonomous driving, as "Grand Challenges." It examines the factors that make these problems particularly significant and challenging in the field of AI.
- **Designing Effective Game-Playing Systems**: This problem investigates the fundamental components and strategies involved in designing AI systems capable of playing games effectively. It explores the interplay between knowledge representation, planning, and decision-making in game-playing algorithms.
- **Understanding the Architecture of Advanced AI Systems**: This problem explores the architecture and design of complex AI systems, such as AlphaGo, which combine deep learning and search techniques. It examines how these systems integrate different components to achieve high performance in challenging tasks.
- **Representing Linear Transformations with Matrices**: Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic**: The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations**: Systems of linear equations can be represented and solved using matrices and their inverses. This is crucial for various AI applications.
- **Matrix Transposition**: Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix. It's used in various AI algorithms for data manipulation.
- **Understanding Perceptrons**: The perceptron, a fundamental building block of neural networks, is introduced. Its structure and function are explained using a diagram.
- **Parallelizing Matrix Multiplication**: Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Calculus in AI**: The slide briefly mentions the importance of calculus, specifically derivatives, partial derivatives, and finding maxima/minima/gradients, in AI algorithms.
- **Calculating the Derivative of the Sigmoid Function**: The slide shows the step-by-step derivation of the derivative of the sigmoid function using calculus rules. This is important for training neural networks.
- **Linear Regression**: Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression**: Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting**: Fitting a curve to data points, potentially using polynomial functions of higher degrees. This can lead to overfitting if the model is too complex.
- **Binary Classification**: Classifying data points into two categories (+1 or -1) using a linear model and a loss function.
- **Bag of Features Object Categorization**: This problem involves using Bag of Features to categorize objects by mapping their feature vectors into a model space.
- **Edge Detection**: This problem focuses on identifying edges in images, which are curves where brightness changes significantly.

- **Corner Detection**: This problem involves identifying corners in images, which are points where the image's intensity changes significantly in multiple directions.
- **Vision Pyramids**: This problem focuses on representing an image as a "pyramid" of progressively smaller and blurred versions to extract features at different scales.
- **Object Detection and Recognition using CNNs**: Convolutional Neural Networks are used to identify and locate objects within images and videos. Examples include facial landmark detection, car detection in highway scenes, and fruit recognition.
- **Improving ImageNet Classification Accuracy**: This problem focuses on improving the accuracy of image classification models over time, as demonstrated by the decreasing error rate in the ImageNet challenge. AlexNet is highlighted as a significant milestone.
- **Comparing Dense and Convolutional Neural Networks**: This problem involves understanding the architectural differences between standard fully connected neural networks and convolutional neural networks (CNNs), particularly concerning their connectivity patterns and data processing in multiple dimensions.
- **Understanding Artificial Neurons**: This problem involves understanding the function of an artificial neuron, including weighted input summation, optional activation functions (like sigmoid and ReLU), and their role in introducing non-linearity.
- **Convolution and Pooling Operations in CNNs**: This problem focuses on understanding the mathematical operations of convolution and pooling layers in CNNs, including the application of convolution filters (like Sobel Gx) and pooling functions (like max pooling) to reduce spatial dimensions.
- **Explaining Sparse Connectivity in Convolutional Layers**: This problem involves understanding why convolutional layers use sparse connectivity instead of dense connectivity, focusing on the concepts of local receptive fields and parameter sharing.
- **Understanding the ReLU Activation Function**: This problem involves understanding the rectified linear unit (ReLU) activation function, its mathematical definition ($f(x) = \max(0, x)$), and its graphical representation.
- **Understanding Pooling Layers**: This problem involves understanding the function of pooling layers in CNNs, including max pooling and average pooling, and their role in downsampling the input volume.
- **Backpropagation for Neural Network Training**: This problem involves understanding the backpropagation algorithm used to train neural networks by adjusting weights to minimize the difference between predicted and desired outputs.
- **Understanding a Simple CNN Structure**: This problem involves understanding the architecture of a simple CNN, including the sequence of convolutional, ReLU, pooling, and fully connected layers, and their roles in image classification.
- **Analyzing Learned Convolution Filters**: This problem involves analyzing the learned convolution filters from a CNN, observing the patterns and textures they have learned to detect.
- **Multidimensional Convolution**: This problem involves understanding how multidimensional convolution operates on images with multiple color channels (e.g., RGB).
- **Biological Inspiration for Artificial Neural Networks**: This problem involves understanding the biological inspiration behind artificial neural networks, drawing parallels between biological neurons and perceptrons.
- **Using the MNIST Dataset**: This problem involves understanding the MNIST dataset, its characteristics (handwritten digits, size normalization), and its use in training and testing machine learning models.
- **LeNet-5 Architecture for MNIST**: This problem involves understanding the LeNet-5 architecture, its layers (convolutional, pooling, fully connected), and how it processes the MNIST dataset.
- **CIFAR-10 Dataset and State-of-the-Art Models**: This problem involves understanding the CIFAR-10 dataset, its characteristics (32x32 color images, 10 classes), and the state-of-the-art accuracy achieved by various deep learning models on this dataset.
- **Generalization and Overfitting**: This problem focuses on the ability of a model to generalize to unseen data, and the risk of overfitting to the training data. The concepts are illustrated using cartoon robots representing well-generalized, well-fitted, and overfitted models.
- **Training \textbackslash{}& testing errors**: This problem involves analyzing the prediction error on training and testing datasets to understand the bias-variance tradeoff and diagnose overfitting or

underfitting. A graph visually represents the relationship between model complexity and prediction error for both training and testing samples.

- **Training and Testing**: This problem addresses the process of splitting data into training, held-out, and test sets to evaluate a machine learning model's performance and generalization ability. Cartoon robots are used to illustrate the training, practice exam, and final exam phases.
- **Hyper-Parameter Tuning**: This problem involves finding the optimal hyperparameters for a machine learning model to improve its performance. A cartoon robot is used to represent the process of adjusting hyperparameters.
- **Fit a cubic function with a degree-9 polynomial**: This problem demonstrates overfitting by fitting a high-degree polynomial to data, resulting in a model that fits the training data well but generalizes poorly. Three graphs illustrate the effect of increasing polynomial degree on the fit.
- **Control the Model Complexity**: This problem explores methods for controlling model complexity, such as using the degree of a polynomial or the magnitude of weights in a model. A table shows weights for polynomials of different degrees.
- **Regularized Linear Regression**: This problem introduces a method to prevent overfitting in linear regression by adding a regularization term to the loss function. The mathematical formulation of the regularized loss function is presented.
- **Tuning on Held-Out Validation Data**: This problem focuses on using a held-out validation set to tune hyperparameters and prevent overfitting. A graph shows how different hyperparameter values affect accuracy on training, validation, and test sets.
- **Learning curves illuminate likely causes of error**: This problem uses learning curves (graphs of accuracy vs. model complexity) to diagnose underfitting and overfitting. Graphs show the relationship between model complexity and accuracy on training and test sets.
- **S-fold cross validation**: This problem describes a method for evaluating model performance and selecting hyperparameters using cross-validation. A diagram illustrates the process of splitting data into training and validation sets for each fold.
- **The Data Analysis Pipeline**: This problem outlines the stages of a data analysis pipeline, highlighting potential issues at each stage. A grid of diverse figures represents different data types and models.
- **Matrix Factorization**: Approximating a high-dimensional data matrix as a product of two lower-dimensional matrices to reveal latent factors.
- **Dimensionality Reduction**: Transforming data from a higher number of dimensions to a lower number of dimensions.
- **Low Rank Matrix Approximations**: Reducing the rank of a matrix to achieve dimensionality reduction.
- **Singular Value Decomposition (SVD)**: Decomposing a matrix into three simpler matrices (U, S, V) to reveal latent factors.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data.
- **Collaborative Filtering**: Predicting a user's rating for an item based on the ratings of similar users.
- **Recommender System Algorithms**: Inferring missing ratings in a user-item matrix based on the ratings of similar users.
- **Modeling Systematic Biases**: Accounting for biases in user ratings and movie ratings to improve the accuracy of recommender systems.
- **Bias-Variance Tradeoff**: Balancing model complexity (number of parameters) and prediction accuracy (RMSE) in recommender systems.
- **Clustering Data**: Grouping a set of objects into classes of similar objects, where objects within a cluster are similar and objects from different clusters are dissimilar. This is a common form of unsupervised learning.
- **Supervised vs. Unsupervised Learning**: Supervised learning uses paired inputs/outputs to learn a function, while unsupervised learning uses only inputs to learn a function to simplify the data. Examples of unsupervised learning include clustering and matrix factorization.
- **Choosing the Number of Clusters (k) in K-means**: Determining the optimal number of clusters (k) in K-means clustering by minimizing the sum of squared distances from each point to its cluster centroid (inertia). A visual approach involves finding the "elbow" in a plot of inertia versus k.

- **Evaluating Clustering Results Subjectively**: Assessing the quality of clusters by visually inspecting a reordered proximity matrix (heatmap) for a clear block pattern, or by using PCA to reduce dimensionality and visualizing the clusters in a lower-dimensional space.
- **Evaluating Clustering Results Objectively (with Class Labels)**: Measuring the quality of clusters when ground truth class labels are available using metrics like purity or by comparing an observed similarity matrix (based on cluster membership) to an ideal similarity matrix (based on class labels).
- **Handling Non-Ideal Data in K-means**: Addressing the limitations of K-means clustering when the data does not meet its assumptions (e.g., non-spherical clusters, unequal variances, unevenly sized clusters).
- **Solving the Satisfiability Problem (SAT)**: Determining whether a given propositional formula is satisfiable, meaning there exists an assignment of truth values to its variables that makes the formula true.
- **Converting Logical Expressions to Conjunctive Normal Form (CNF)**: Transforming arbitrary logical expressions into an equivalent CNF representation, which is a conjunction of clauses, each being a disjunction of literals.
- **Model Checking using SAT**: Utilizing SAT solvers to verify properties of systems, such as safety and liveness, by encoding the system's behavior and properties into a CNF formula.
- **Classical Planning with SAT**: Employing SAT solvers to find optimal plans by encoding the planning problem's constraints and goals into a CNF formula.
- **Solving Combinatorial Design Problems with SAT**: Applying SAT solvers to find solutions to complex mathematical structures with desirable properties, such as Latin squares or Steiner systems.
- **Applying SAT to Diverse Domains**: Using SAT solvers to address sub-problems in various fields, including test pattern generation, scheduling, optimal control, protocol design, and multi-agent systems.
- **Reducing SAT to 3-SAT**: Transforming a general SAT problem into an equivalent 3-SAT problem, where each clause contains exactly three literals, by introducing new variables.
- **Representing Recursion**: This problem focuses on understanding and implementing recursive algorithms and data structures, such as the Fibonacci sequence and tree traversal.
- **Problem Solving as Search**: This problem involves formulating problems as search problems, defining states, actions, and goal tests, and choosing appropriate search algorithms. Examples include mazes and the 8-puzzle.
- **Optimizing Recursive Algorithms**: This problem addresses the efficiency of recursive algorithms by identifying and eliminating redundant calculations, as demonstrated with the Fibonacci sequence.
- **Searching Graphs and Maps**: This problem extends search algorithms to graph-based structures, considering multiple paths to a solution and the computational complexity of searching large state spaces. Examples include pathfinding on a campus map and the traveling salesperson problem.
- **Defining Rational Agents**: This problem focuses on understanding the concept of rational agents in AI, where the goal is to design agents that maximize goal achievement given available information.
- **Improving Reasoning and Modeling Technology**: This problem focuses on developing better technology for reasoning and modeling, aiming to improve the efficiency and effectiveness of automated reasoning systems.
- **Solving the Boolean Satisfiability Problem (SAT)**: This problem involves determining whether there exists an assignment of truth values to variables that satisfies a given Boolean formula in Conjunctive Normal Form (CNF).
- **Addressing Exponential Complexity Growth**: This problem addresses the challenge of exponential complexity growth in various problem domains, particularly in propositional reasoning, and explores strategies to mitigate this challenge.
- **Bounded Model Checking (BMC)**: This problem involves verifying whether a given model satisfies a temporal property within a specified bound, often reduced to a SAT problem.
- **Encoding Problems into SAT**: This problem focuses on translating various problem instances from different domains (e.g., planning, verification, combinatorial design) into a SAT formula that can be solved by a SAT solver.
- **Graph Coloring**: This problem involves assigning colors to vertices in a graph such that no two adjacent vertices share the same color.
- **Hamiltonian Path Problem**: This problem involves determining whether a path exists in a directed

graph that visits each vertex exactly once.
- **Data Bias and Fairness**: This problem focuses on the presence of bias and unfairness in data used to train AI models, leading to discriminatory outcomes. Examples include bias in loan applications, hiring processes, and criminal justice risk assessments.
- **Data Privacy and Anonymization**: This problem explores the challenges of protecting individual privacy when using personal data for AI applications, even when data is anonymized. Examples include re-identification of individuals from seemingly anonymized datasets and the inherent identifiability of certain data types like genome sequences.
- **Algorithmic Transparency and Explainability**: This problem addresses the difficulty of understanding how complex AI models arrive at their decisions, making it hard to trust or debug them. The lack of transparency can hinder efforts to identify and mitigate bias.
- **Defining and Measuring Fairness in AI**: This problem focuses on the lack of consensus on how to define and measure fairness in AI systems. Different notions of fairness (e.g., group fairness, individual fairness) may conflict, making it challenging to develop fair and equitable AI systems.
- **The Risk of Spurious Correlations**: This problem highlights the risk of AI models learning spurious correlations from data, leading to inaccurate or misleading conclusions. An example is a study showing that asthmatic patients had a lower risk of dying from pneumonia due to aggressive care, but this correlation doesn't imply causation.
- **Addressing Algorithmic Bias**: This problem focuses on developing methods to detect, mitigate, and remove bias from AI models. Strategies include constraining the algorithm to ignore protected attributes, carefully curating training data, and post-processing techniques to remove learned biases.

3 Algorithm Solutions

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally. It uses an evaluation function to assign scores to game states and recursively explores the game tree, alternating between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning**: A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha (best value for the maximizing player) and beta (best value for the minimizing player) values to prune branches.
- **Matrix Addition**: $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication**: $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication**: $C_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj}$
- **Matrix Inverse**: $AA^{-1} = A^{-1}A = I$
- **Solving Linear Equations using Matrix Inverses**: $Ax = b \implies x = A^{-1}b$
- **Derivative of Sigmoid Function**: $\frac{d}{dx}\sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$
- **Perceptron Algorithm**: Iteratively update the weight vector w by adding $y_n x_n$ if the prediction $\text{sign}(w^T x_n)$ is incorrect. If the data is linearly separable, this algorithm will find the separating hyperplane.
- **Logistic Regression**: Minimize the logistic loss function $F(w) = \sum_{i=1}^{N} \log(1 + e^{-y_i w^T x_i})$ using stochastic gradient descent. The update rule is $w \leftarrow w + \alpha(\sigma(-y_n w^T x_n) y_n x_n)$, where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function.
- **Bag of Features**: Represent images by frequencies of "visual words" (Bags of features)
- **Nearest Neighbor Classifier**: Assign label of nearest training data point to each test data point
- **Moving Average**: Replace each pixel with an average of all the values in its neighborhood
- **Weighted Moving Average**: Add weights to our moving average
- **Image Sharpening**: $f_{sharp} = f + \alpha(f f_{blur})$

- **Finite Difference**: $\frac{df}{dx}[x,y] \approx F[x+1,y] - F[x,y]$
- **Image Gradient Magnitude**: $||\nabla f|| = \sqrt{(\frac{\partial f}{\partial x})^2 + (\frac{\partial f}{\partial y})^2}$
- **Image Gradient Direction**: $\theta = \tan^{-1}(\frac{\partial f}{\partial x} / \frac{\partial f}{\partial y})$
- **Sum of Squared Differences (SSD)**: $E(u,v) = \sum_{(x,y)\in W} [I(x+u, y+v) - I(x,y)]^2$
- **Anisotropic Gaussian**: $G_{\sigma_x,\sigma_y}(x,y) = \frac{1}{2\pi\sigma_x\sigma_y} \cdot e^{-\frac{1}{2}((\frac{x^2}{\sigma_x^2})+(\frac{y^2}{\sigma_y^2}))}$
- **Isotropic Gaussian**: $G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \cdot e^{-\frac{r^2}{2\sigma^2}}$
- **ReLU Activation Function**: $f(x) = \max(0,x)$
- **Convolution Operation**: Element-wise multiplication of a filter with a section of the input image, followed by summation of the results.
- **Max Pooling**: Selecting the maximum value within a defined region (e.g., 2x2) of the input.
- **Average Pooling**: Calculating the average value within a defined region (e.g., 2x2) of the input.
- **Backpropagation**: An algorithm that calculates the gradient of the loss function with respect to the network's weights and uses this gradient to update the weights, iteratively minimizing the loss.
- **Regularized Linear Regression**: $E(w) = RSS(w) + \lambda R(w)$, where $R(w) = ||w||_2^2 = w_1^2 + ... + w_k^2$ or $R(w) = ||w||_1 = |w_1| + ... + |w_k|$.
- **S-fold Cross Validation**: Split training data into S equal parts; use each part as validation data, others as training data; choose hyperparameter with best average performance.
- **Singular Value Decomposition (SVD)**: $A = USV^T$, where A is an $n \times d$ matrix, U is an $n \times n$ orthogonal matrix, V is a $d \times d$ orthogonal matrix, and S is an $n \times d$ diagonal matrix with nonnegative entries sorted from high to low.
- **Principal Component Analysis (PCA)**: A linear dimensionality reduction method that finds the directions (principal components) that maximize the variance of the data. The eigenvectors of the covariance matrix correspond to the principal components.
- **Modeling Systematic Biases in Recommender Systems**: $r_{ij} \approx \mu + b_i + b_j + u_i \cdot v_j$, where μ is the overall mean rating, b_i is the bias for user i, b_j is the bias for movie j, and $u_i \cdot v_j$ represents the user-movie interaction.
- **Root Mean Square Error (RMSE)**: $\frac{1}{|R|} \sum_{(u,i)\in R} (r_{ui} - \hat{r}_u)^2$, where r_{ui} is the actual rating and $\hat{r}_u$ is the predicted rating.
- **K-means Clustering**: The algorithm iteratively assigns data points to the nearest cluster centroid, then recomputes the centroids as the mean of the points in each cluster, repeating until convergence.
- **Euclidean Distance**: $d_E(x,y) = \sqrt{\sum_{i=1}^{p} (x_i - y_i)^2}$, where x and y are p-dimensional vectors.
- **Hamming Distance**: Counts the number of differing bits between two binary vectors.
- **Matching Coefficient**: Hamming distance normalized by the number of attributes/bits.
- **Purity**: $\text{purity}(C,G) = \frac{1}{N} \sum_{i=1}^{K} \max_j N_j \in G_i$, where N is the total number of data points, K is the number of clusters, and N_j is the number of points of class j in cluster i.
- **SAT**: Determining whether a given propositional formula is satisfiable.
- **3-SAT**: Reducing the unrestricted SAT problem to 3-SAT by transforming each clause $l_1 \vee ... \vee l_n$ to a conjunction of $n-2$ clauses. The resulting formula is equisatisfiable with the original, meaning they have the same set of satisfying assignments.
- **CNF Conversion**: A set of rules to transform logical expressions into Conjunctive Normal Form (CNF). The rules define how to convert various logical operators (implication, conjunction, disjunction, biconditional, negation) into CNF.
- **Recursive Problem Solving**: Do part of the work and reduce the rest into a similar but smaller

problem.

- **Structural Recursion**: Traversal of data structures like trees using recursion. Breadth-First Search visits nodes level by level; Depth-First Search traverses down a branch.
- **Structural Recursion (Binary Tree Example)**: Searching a binary search tree recursively. If the current node's value (V) equals the target value (X), return True. If $X < V$, search the left subtree; if $X > V$, search the right subtree. If there are no children, return False.
- **Searching Graphs and Maps**: Finding paths in graphs and maps using search algorithms. States are nodes, actions are edges, and the goal is to find a path from the start node to the goal node.
- **8-Puzzle as a Graph**: Solving the 8-puzzle by representing it as a graph. States are tile configurations, actions are tile movements, and the goal is to reach the sorted configuration. Avoid exploring the same state twice.
- **Robotic Assembly**: Representing robotic assembly as a search problem. States are joint angles and part positions, actions are joint movements, the goal is complete assembly, and the path cost is assembly time.
- **Simple Search Abstraction Assumptions**: Static (world doesn't change), Discrete (finite states), Observable (states are known), Deterministic (actions have certain outcomes).
- **Pac-Man as an Agent**: Modeling Pac-Man as an agent interacting with an environment. The agent receives percepts from sensors, performs actions through actuators, and aims to achieve a goal (e.g., eating all dots).
- **Naïve\textbackslash{}\textbackslash{}_SAT**: A recursive algorithm that explores all possible variable assignments to determine the satisfiability of a propositional formula. It checks for immediate satisfiability or unsatisfiability, then recursively calls itself with assignments of true and false to an unassigned variable, backtracking if both fail.
- **DPLL**: A recursive algorithm that improves upon the Naïve\\$_SATalgorithmbyincorporatingBooleanConstraintPropaga$
$Asub-algorithmwithinDPLLthatrepeatedlyidentifiesandassignsvaluestoliteralsinunitclauses(clauseswithonlyoneun$

- **Resolution Rule**: An inference rule that derives a new clause from two clauses containing a literal and its negation. The new clause combines the remaining literals from the original clauses using logical OR.
- **Clause Learning**: A technique used to learn from conflicts encountered during the DPLL search. When a conflict occurs, a concise reason (a clause) is learned to prevent similar conflicts in the future, improving efficiency.
  - **Fairness through Blindness**: Ignore all irrelevant/protected attributes. However, it is possible to predict these attributes with high accuracy from other data.
  - **Group Fairness**: $P(\text{outcome } o|S) = P(\text{outcome } o|T)$, where S is a minority group and T is the rest of the population. This is not a strong enough notion of fairness.
  - **Individual Fairness**: Treat similar individuals similarly. Similar individuals should be treated similarly for the purpose of the classification task and should be similarly distributed over outcomes. How to achieve this is an active area of research.